

VII. LES REQUETES IMBRIQUÉES

- INTRODUCTION
- COMMANDE SELECT
- SELECT-PROJECTION
- SELECT-SÉLECTION
- SELECT-COMPARAISON À UNE VALEUR
- SELECT-COMPARAISON À UNE FOURCHETTE DE VALEUR
- SELECT-COMPARAISON À UNE LISTE DE VALEURS
- SELECT-COMPARAISON À UN FILTRE
- TRI DES TUPLES
- CLAUSE GROUP BY
- FONCTIONS DE GROUPE
- FONCTION COUNT
- FONCTION SUM
- FONCTION AVG
- FONCTIONS MIN ET MAX
- CLAUSE HAVING
- CLAUSE LIMIT
- COMMANDE INSERT
- COMMANDE UPDATE
- COMMANDE DELETE

Introduction

❖ Une **requête imbriquée (Nested query) ou sous-requête (subquery)** est une requête qui est **incluse à l'intérieur d'une autre requête**.

❖ Il est possible d'imbriquer plusieurs requêtes et il n'y a pas de limite dans le nombre de niveaux d'imbrication

❖ Une requête imbriquée peut être faite dans une requête de type SELECT, INSERT, UPDATE ou DELETE

❖ Elle peut être utilisée dans les différentes clauses et commandes:

- FROM
- WHERE (SELECT, UPDATE, DELETE)
- SET (UPDATE)
- INSERT INTO (INSERT)
- CREATE
- On utilise les requêtes imbriquées **lorsque les jointures ne sont pas possibles**

Sous-requête dans la clause WHERE

- ❖ Ce type de **sous-requête** permet de **comparer** une valeur de la clause **WHERE** avec le résultat retourné par une sous-requête.
- ❖ La sous-requête peut retourner deux types de résultats
 - Résultat sur **1 ligne**
 - Résultat sur **plusieurs lignes**

Sous-requête dans la clause WHERE

❖ Requête retournant une seule ligne

- Lorsque la sous-requête retourne un résultat sur **une seule ligne**, on utilise les opérateurs suivants: =, !=, <, <=, >, >=

- **Syntaxe:**

SELECT colonne1, colonne2, ...

FROM nom_tableA

WHERE colonnex OP (SELECT colonnex
FROM nom_tableB
...
)

Suite_requête

- Où OP est =, !=, <, <=, >, >=

Sous-requête dans la clause WHERE

❖ Requête retournant une seule ligne

- Soit la table syemp(empo, ename, sal, deptno)
- **EX1:** Le nom, ename des employés qui travaillent dans le même département que FORD

SELECT ename

FROM syemp

WHERE deptno = (SELECT deptno
FROM syemp
WHERE ename = 'FORD');

syemp			
EMPNO	ENAME	SAL	DEPTNO
7839	KING	5000	10
7698	BLAKE	2850	30
7782	CLARK	2450	10
7566	JONES	2975	20
7902	FORD	3000	20
7369	SMITH	800	20
7499	ALLEN	1600	30
7521	WARD	1250	30
7654	MARTIN	1250	30
7844	TURNER	1500	30
7900	JAMES	950	30
7934	MILLER	1300	10

Sous-requête dans la clause WHERE

❖ Requête retournant une seule ligne

- Soit la table syemp(empo, ename, sal, deptno)
- **EX2:** Le nom de l'employé qui a le salaire le plus élevé. suivi de ce salaire.

SELECT ename, salaire

FROM syemp

WHERE sal = (SELECT max(sal)
FROM syemp) ;

syemp			
EMPNO	ENAME	SAL	DEPTNO
7839	KING	5000	10
7698	BLAKE	2850	30
7782	CLARK	2450	10
7566	JONES	2975	20
7902	FORD	3000	20
7369	SMITH	800	20
7499	ALLEN	1600	30
7521	WARD	1250	30
7654	MARTIN	1250	30
7844	TURNER	1500	30
7900	JAMES	950	30
7934	MILLER	1300	10

Sous-requête dans la clause WHERE

❖ Requête retournant plusieurs lignes

- Lorsque la sous-requête retourne un résultat sur **plusieurs lignes**, on utilise les opérateurs suivants: **IN, NOT IN, ALL, ANY**

- **Syntaxe:**

SELECT colonne1, colonne2, ...

FROM nom_tableA

WHERE exp **IN** | exp **NOT IN** | exp **op ANY** | exp **op ALL** (**SELECT ...**)

- Où **op** est **=, !=, , <=, >=**

Sous-requête dans la clause WHERE

❖ Requête retournant plusieurs lignes

- **IN** : Permet de tester si une valeur donnée existe dans la liste retournée par la sous-requête
- **ANY** : La requête principale est exécutée si la condition est vraie pour au moins un élément de la sous-requête
- **ALL** : La requête principale est exécutée si la condition est vraie pour toutes les valeurs retournées par la sous-requête

Sous-requête dans la clause WHERE

❖ Requête retournant plusieurs lignes

- Soit la table syemp(empo, ename, sal, deptno)
- **EX1:** On cherche le nom des employés qui travaillent dans le même département que l'employé FORD ou l'employé MILLER

```
SELECT ename  
FROM syemp  
WHERE deptno IN ( SELECT deptno  
                  FROM syemp  
                  WHERE ename ='FORD'  
                  OR ename ='MILLER' );
```

syemp			
EMPNO	ENAME	SAL	DEPTNO
7839	KING	5000	10
7698	BLAKE	2850	30
7782	CLARK	2450	10
7566	JONES	2975	20
7902	FORD	3000	20
7369	SMITH	800	20
7499	ALLEN	1600	30
7521	WARD	1250	30
7654	MARTIN	1250	30
7844	TURNER	1500	30
7900	JAMES	950	30
7934	MILLER	1300	10

Sous-requête dans la clause WHERE

❖ Requête retournant plusieurs lignes

- Soit la table syemp(empo, ename, sal, deptno)
- **EX2:** Noms des employés du département numéro 10 dont le salaire est plus grand que celui de TOUS les employés du département 30

SELECT ename

FROM syemp

WHERE deptno =10 AND sal > **ALL** (SELECT sal
FROM syemp
WHERE deptno=30);

syemp			
EMPNO	ENAME	SAL	DEPTNO
7839	KING	5000	10
7698	BLAKE	2850	30
7782	CLARK	2450	10
7566	JONES	2975	20
7902	FORD	3000	20
7369	SMITH	800	20
7499	ALLEN	1600	30
7521	WARD	1250	30
7654	MARTIN	1250	30
7844	TURNER	1500	30
7900	JAMES	950	30
7934	MILLER	1300	10

Sous-requête dans la clause WHERE

❖ Requête retournant plusieurs lignes

- Soit la table syemp(empo, ename, sal, deptno)
- **EX3:** Noms des employés du département numéro 10 dont le salaire est plus grand que celui de n'importe quel employés du département 30

SELECT ename

FROM syemp

WHERE deptno =10 AND sal > **ANY** (SELECT sal
FROM syemp
WHERE deptno=30);

syemp			
EMPNO	ENAME	SAL	DEPTNO
7839	KING	5000	10
7698	BLAKE	2850	30
7782	CLARK	2450	10
7566	JONES	2975	20
7902	FORD	3000	20
7369	SMITH	800	20
7499	ALLEN	1600	30
7521	WARD	1250	30
7654	MARTIN	1250	30
7844	TURNER	1500	30
7900	JAMES	950	30
7934	MILLER	1300	10

Sous-requête dans la clause FROM

❖ Parfois, on a besoin d'extraire des informations à partir des résultats calculés, ces résultats sont ramenés par une requête SELECT. Dans ce cas nous parlons de requêtes imbriquées dans le **FROM**.

❖ Syntaxe

```
SELECT ...  
FROM ( SELECT ...  
        FROM nom_table  
        ...  
      )  
Suite_requête
```

Sous-requête dans la clause FROM

❖ Soit la table syemp (empo, ename, sal, deptno)

❖ **EX:** Nom des employés ayant les 3 meilleurs salaires

```
SELECT ename  
FROM ( SELECT ename , sal, deptno  
        FROM syemp  
        ORDER BY sal DESC )  
LIMIT 3;
```

syemp			
EMPNO	ENAME	SAL	DEPTNO
7839	KING	5000	10
7698	BLAKE	2850	30
7782	CLARK	2450	10
7566	JONES	2975	20
7902	FORD	3000	20
7369	SMITH	800	20
7499	ALLEN	1600	30
7521	WARD	1250	30
7654	MARTIN	1250	30
7844	TURNER	1500	30
7900	JAMES	950	30
7934	MILLER	1300	10

Sous-requête dans la clause SET

- ❖ On peut chercher des valeurs existantes dans la base de données afin de mettre à jour des données d'une table.
- ❖ La sous-requête est une requête SELECT qui ne doit retourner qu'une seule valeur

- ❖ **Syntaxe**

```
UPDATE nom_table1
SET colonne = ( SELECT ...
                FROM nom_table2
                ... )
Suite_requête;
```

Sous-requête dans la clause SET

❖ Soit la table emp(ename, sal, comm)

❖ **EX:** Mettre à jour les commissions null, par la moyenne de toutes les commissions.

UPDATE emp

SET comm = (SELECT AVG(comm)
FROM emp)

WHERE comm IS NULL ;

emp		
ENAME	SAL	COMM
1 KING	5000	(null)
2 BLAKE	2850	(null)
3 CLARK	2450	(null)
4 JONES	2975	(null)
5 FORD	3000	(null)
6 SMITH	800	(null)
7 ALLEN	1600	300
8 WARD	1250	500
9 MARTIN	1250	1400
0 TURNER	1500	0
1 JAMES	950	(null)
2 MILLER	1300	(null)

Sous-requête dans la commande INSERT INTO

- ❖ Ce type de sous-requête permet d'insérer des données dans une table à partir d'une autre table.
- ❖ La sous requête est utilisée à la place de la clause VALUES de la requête principale et peut retourner plusieurs résultats.
- ❖ **Syntaxe**
`INSERT INTO nom_table_cible (colonne1, colonne2, ...)`
`SELECT colonne1, colonne2, ...`
`FROM nom_table_source`
`WHERE condition;`
- ❖ **NB:** Lors de l'insertion, les **contraintes d'intégrité** doivent être respectées

Sous-requête dans la commande INSERT INTO

- ❖ Soit la table emp(ename, sal, comm)
- ❖ **EX:** Insérer dans la table depart10, les employés du département 10
INSERT INTO depart10 (numemp,nom)
SELECT empno,ename
FROM emp
WHERE deptno =10;

emp		
ENAME	SAL	COMM
1 KING	5000	(null)
2 BLAKE	2850	(null)
3 CLARK	2450	(null)
4 JONES	2975	(null)
5 FORD	3000	(null)
6 SMITH	800	(null)
7 ALLEN	1600	300
8 WARD	1250	500
9 MARTIN	1250	1400
0 TURNER	1500	0
1 JAMES	950	(null)
2 MILLER	1300	(null)

Sous-requête dans la commande CREATE

- ❖ Ce type de requête est utilisé pour créer une table à partir d'une ou plusieurs table existantes.
- ❖ La table créée contient les données issues d'une ou de plusieurs table. • Pour créer une table avec une sous requête.

❖ Syntaxe

```
CREATE TABLE nom_table_cible AS  
SELECT colonne1, colonne2, ...  
FROM nom_table_source  
WHERE condition;
```

Sous-requête dans la commande CREATE

- ❖ Soit la table emp(ename, sal, comm)
- ❖ **EX:** Créer la table depart10 contenant, les employés du département 10
CREATE TABLE depart10 AS
SELECT empno,ename
FROM emp
WHERE deptno =10;

emp		
EMPNO	ENAME	(SAL, COMM)
1	KING	5000 (null)
2	BLAKE	2850 (null)
3	CLARK	2450 (null)
4	JONES	2975 (null)
5	FORD	3000 (null)
6	SMITH	800 (null)
7	ALLEN	1600 300
8	WARD	1250 500
9	MARTIN	1250 1400
0	TURNER	1500 0
1	JAMES	950 (null)
2	MILLER	1300 (null)

Sous-requête avec la clause EXISTS

- ❖ La clause **EXISTS** est utilisée pour tester s'il existe au moins une ligne dans une table satisfaisant une certaine condition.
- ❖ Elle renvoie TRUE si la sous-requête retourne au moins une ligne, FALSE sinon.

- ❖ **Syntaxe**

```
SELECT colonne1, colonne2, ...  
FROM nom_table  
WHERE EXISTS (  
    SELECT 1 | *  
    FROM nom_table_source  
    WHERE condition  
);
```

Sous-requête avec la clause EXISTS

❖ **EX:** La liste de employés qui ont au moins un projet actif

```
SELECT nom  
FROM Employes e  
WHERE EXISTS (  
    SELECT 1  
    FROM Projets p  
    WHERE p.id_employe = e.id AND p.statut = 'actif'  
);
```

Sous-requête avec la clause **HAVING**

- ❖ Un critère dans **HAVING** peut dépendre du résultat d'une requête imbriquées.
- ❖ **Ex:** Affiche le total de jours de stage de chaque catégorie supérieur à la moyenne du nombre de jours de stage

```
SELECT Num-Cat, SUM(Nb-jours) AS somme
FROM STAGE
GROUP BY Num-Cat
HAVING SUM(Nb-jours) > ( SELECT AVG(Nb-jours)
                          FROM STAGE
                        );
```